



Developer Velocity at Scale

Production-Like Airflow Environments on Kubernetes, *Liberty Mutual Insurance*



Matt Davis, Matt Koski

Managed Airflow on Kubernetes doesn't have to mean slow, brittle Airflow development – platform engineering can give teams production-like environments on demand and cut the test-in-production cycle.



Agenda



About Us, the Squad, and Liberty Mutual

Background of Airflow at Liberty Mutual

Challenges with Managed Airflow

Our Solution to Airflow Challenges

Takeaways

Q&A





Matt Davis

- Data Engineer at Liberty Mutual for 6 ½ years
- Worked with Airflow for 3+ years
- Considered “Airflow SME”
- Doc Aggregator
- GitHub enthusiast, AWS Nerd
- Lives in Exeter, NH *“New Hampshire Seacoast”*
- Former high school math teacher

Matt Koski

- Liberty Mutual for 12 years (I think?)
- Runs Kubernetes at work, at home for fun, and sometimes dreams about it
- Lives in Wisconsin
- Likes cheese curds
- Is smart

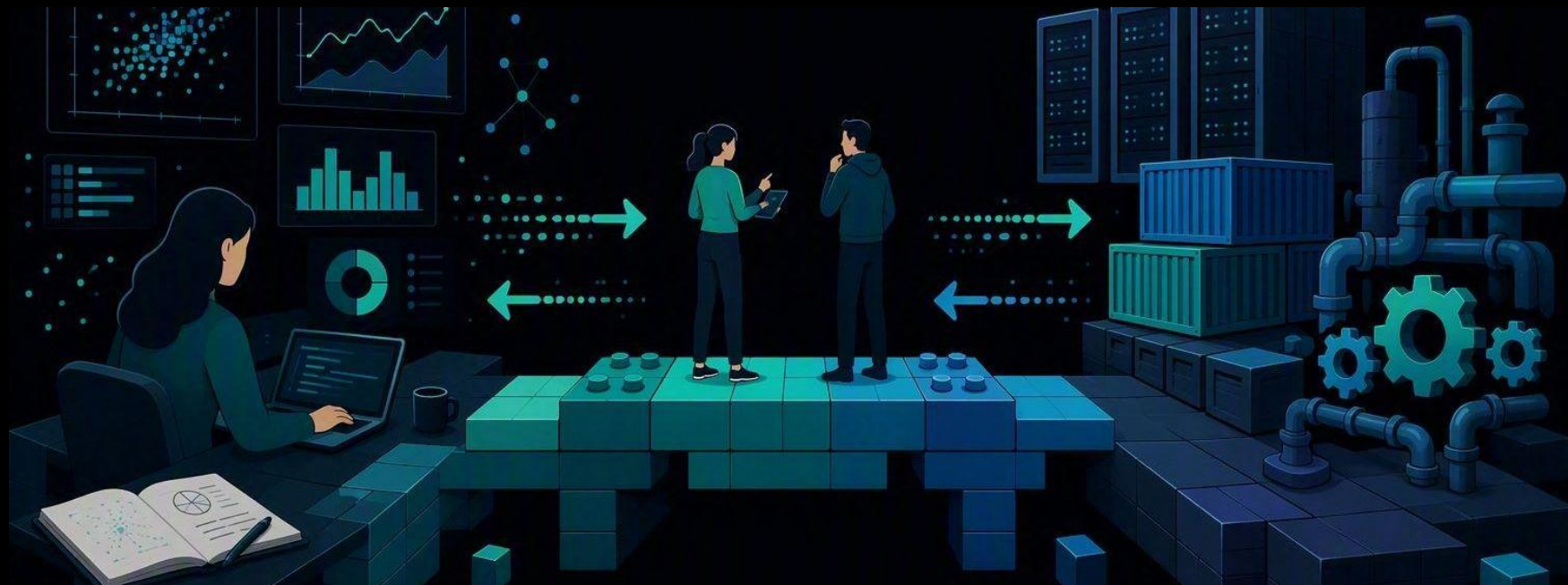
LIBERTY MUTUAL

- Founded in 1912
- Head Quarters – Boston MA
- #95 Fortune Ranking (2025)
- 9th Largest Global Property and Casualty Insurer (2025)
 - Auto/Vehicle
 - Property
 - Casualty
 - Commercial Auto
 - Risk
- Approximately 40,000 Employees Worldwide
- 27 Countries/Economies Around the World

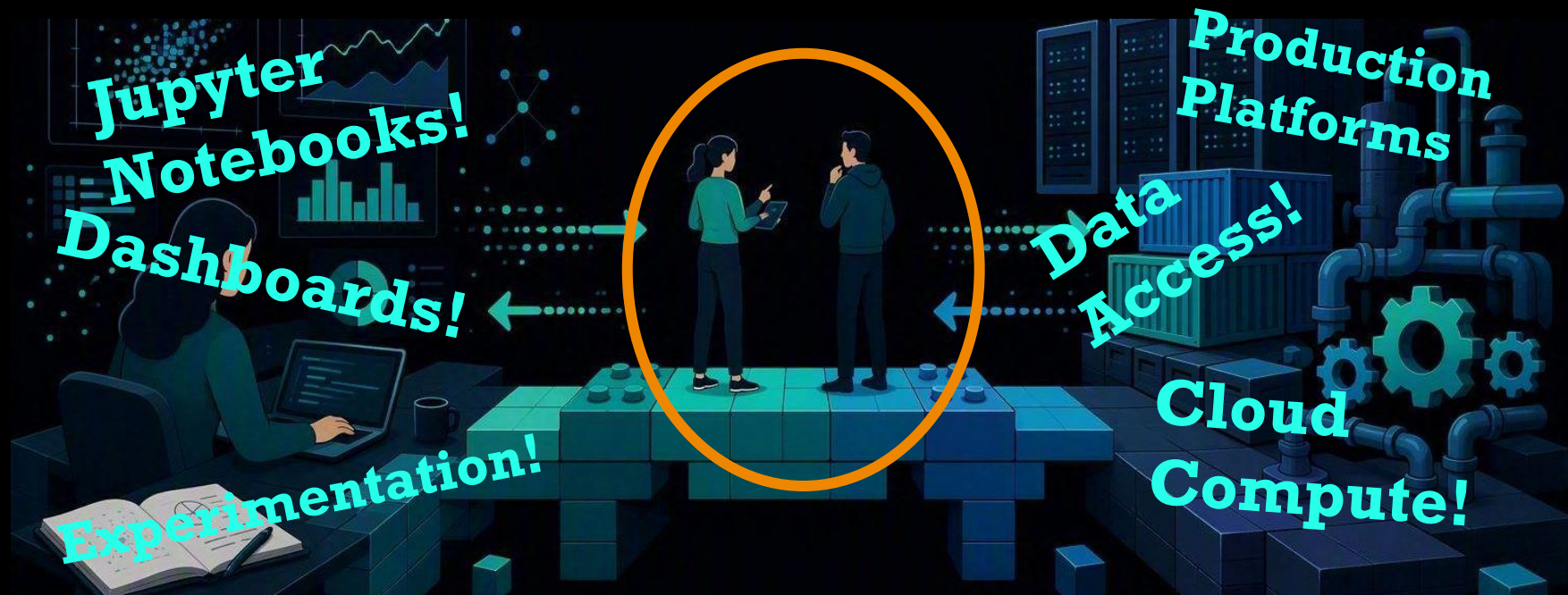




BRIDGING DATA SCIENCE & ENGINEERING



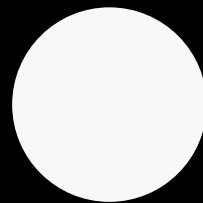
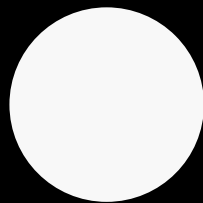
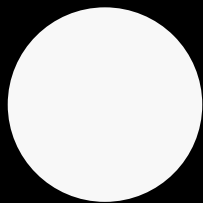
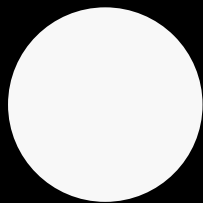
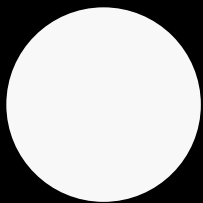
BRIDGING DATA SCIENCE & ENGINEERING



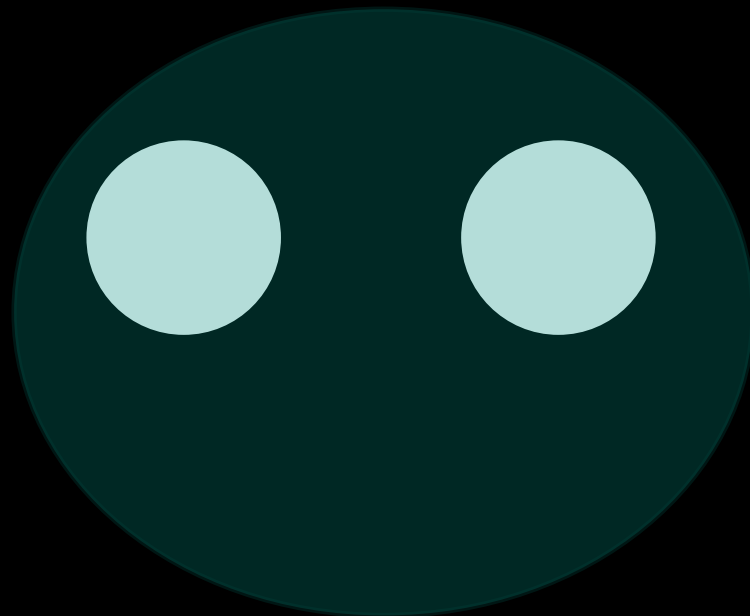
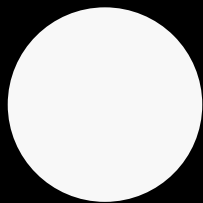
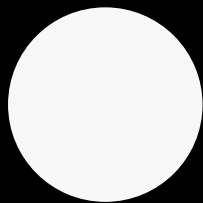
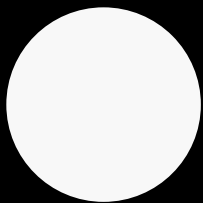


Challenges With Airflow

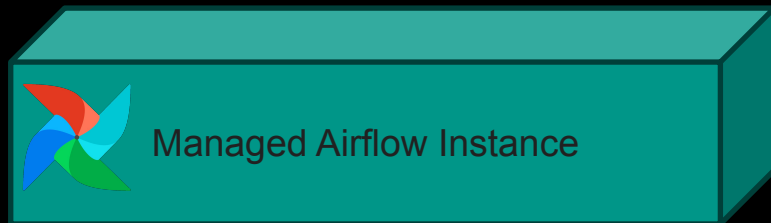
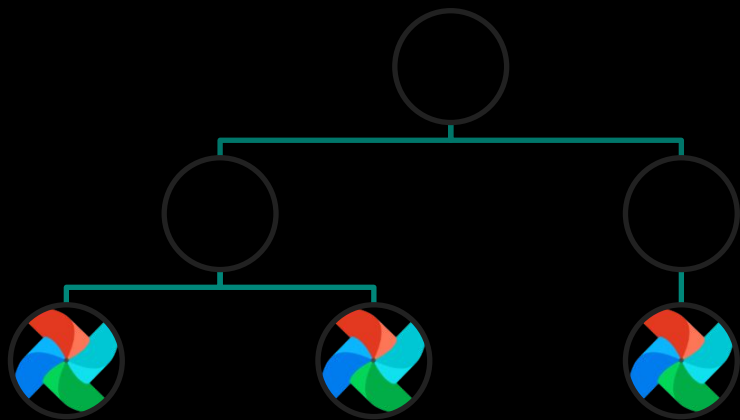
Challenges with Airflow



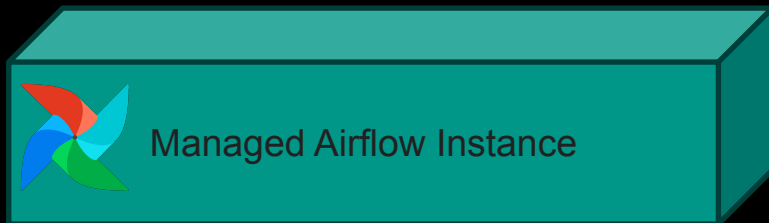
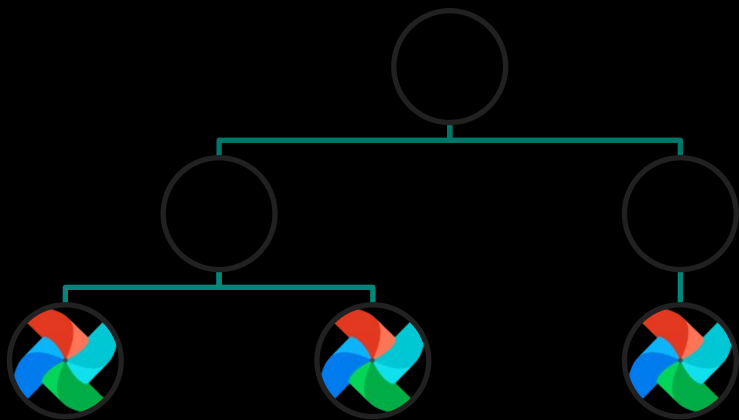
Challenges with Airflow



Team-Wide Development Collisions



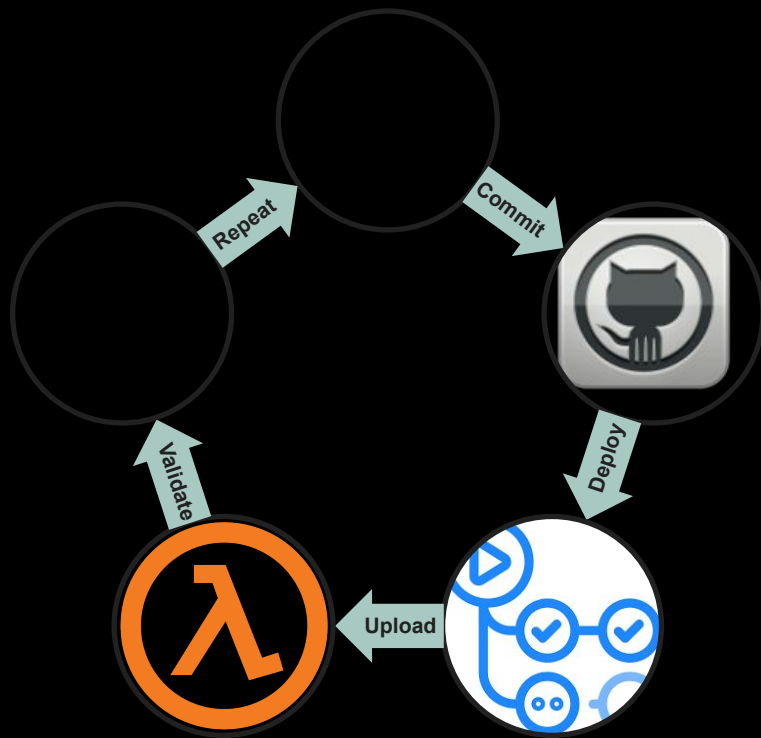
Team-Wide Development Collisions



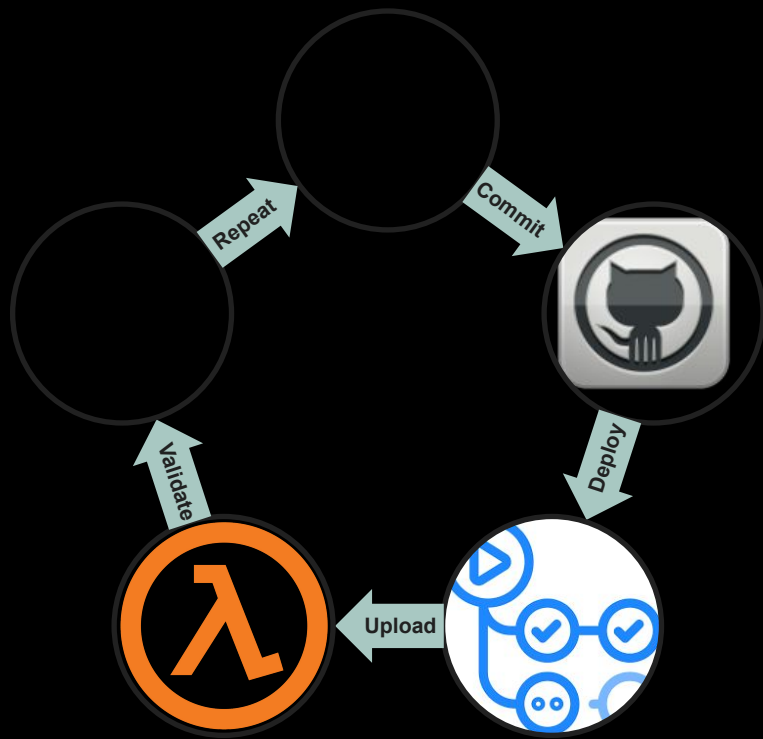
If User A deploys their DAGs to the managed cluster, then User B's DAG would be removed.

If User B deploys their DAG for testing, then User A DAGs will be removed.

Expensive Development Process



Expensive Development Process



Deploying a single DAG can take a significant amount of time.

Minor updates to the DAG which ensure functionality.

Deploy to test and then add more steps as you work through.

How Do DAGs Get to Airflow?



Custom GitHub workflow which bakes in deployment process (AWS) and observability features (Datadog, Splunk)

During deployment, high level syntax checking/validation of users DAGs before entering into Airflow namespace



During deployment, Jinja templating, injecting user variables and pipeline specific values into necessary locales



Engineering team is notified when deployments run off the rails for fast intervention

But What's Your Point in Telling Us This?



But What's Your Point in Telling Us This?

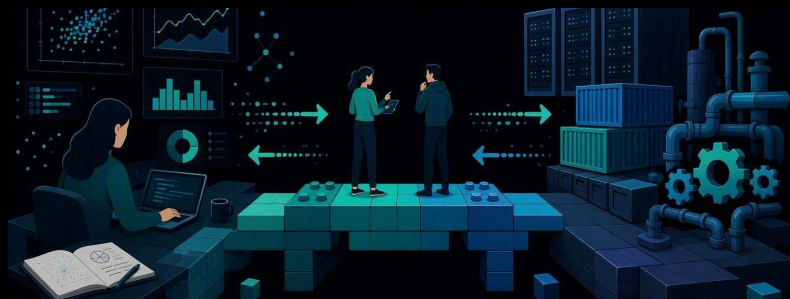


State Library Victoria Collections



Solutions to Airflow Challenges

Leverage What's Already Been Done



User Access

- Easy-to-use UI which customers are already familiar with

Data Access Patterns

- IAM Roles & Service Accounts
- Scoped access to Snowflake/S3/Secrets Manager resources
- Azure groups for secure sharing of application

Stable Infrastructure

- Kubernetes platform for scalability
- Observability via Datadog and Headlamp

Personal Airflow!



1. (Airflow) API Server

2. (Airflow) DAG Processor

3. (Airflow) PostgreSQL

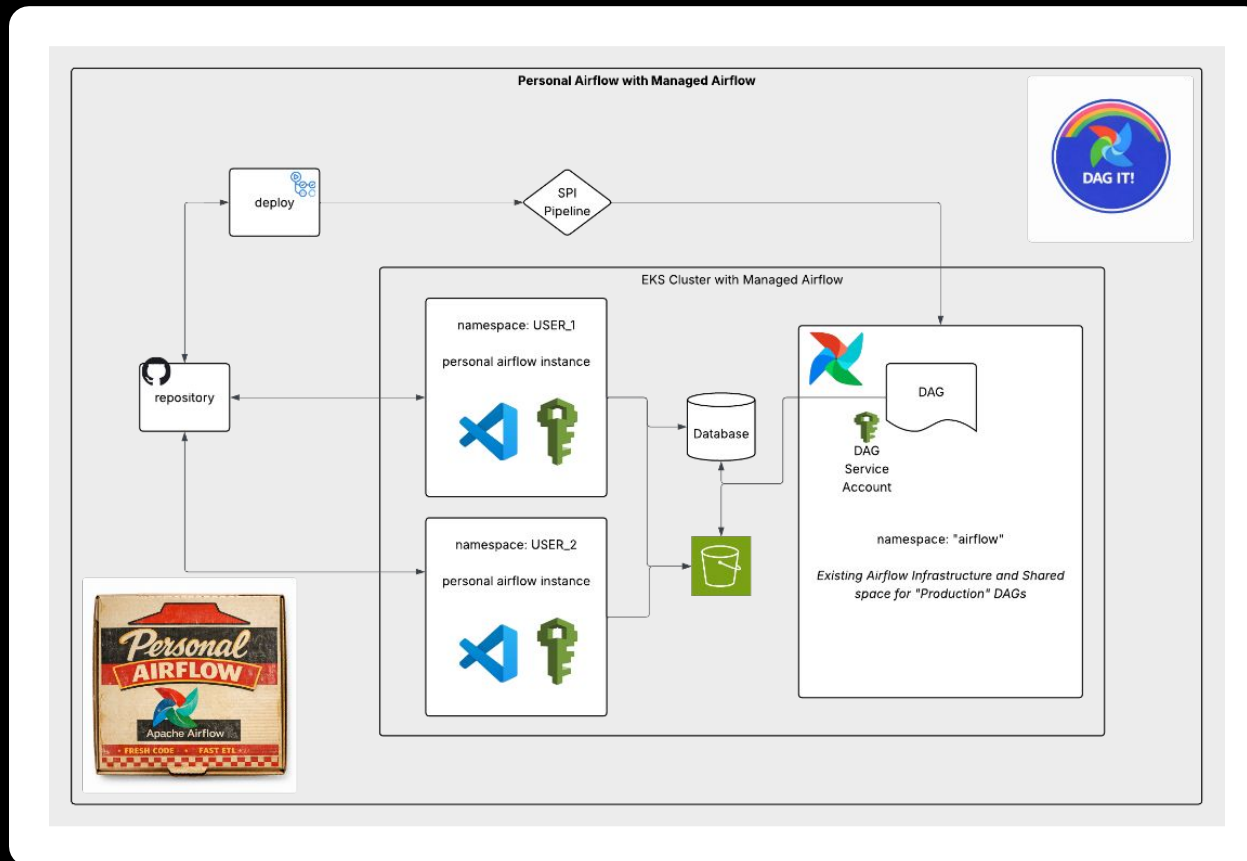
4. (Airflow) Scheduler

5. (Code Server) Auth Sidecar

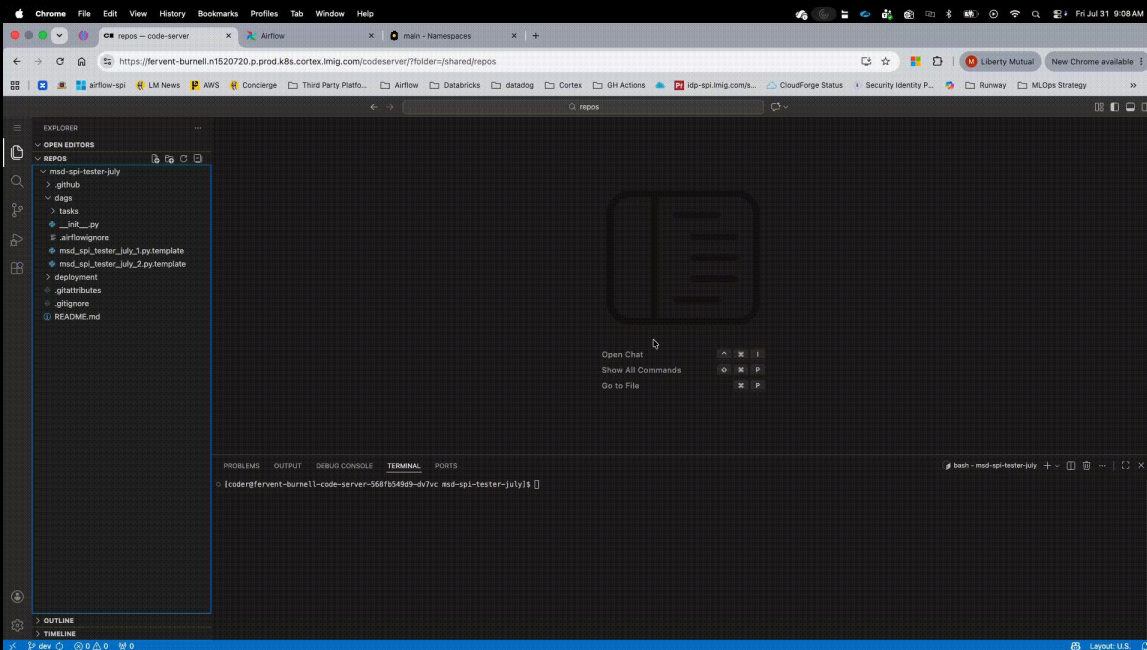
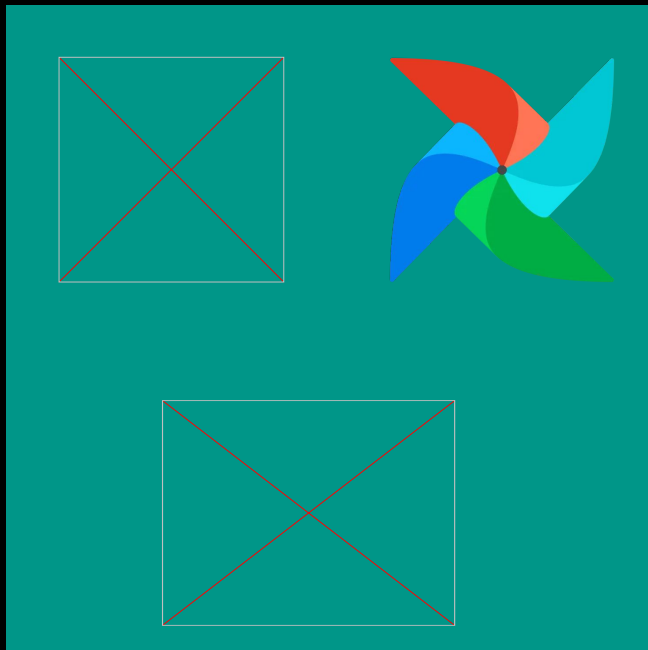
6. (Code Server) VS Code, Copilot

7. (Code Server) DAG Render Sidecar

Kubernetes Implementation



What Does it Look Like?



What do you mean by rendering DAGs?

```
PODSPECS = {  
    "pod_override": k8s.V1Pod(  
        metadata=k8s.V1ObjectMeta(  
            labels={  
                "deployment_guid": "{{ deployment_guid }}",  
                "lm_app": "{{ artifact_key }}",  
                "lm_app_env": "{{ environment }}",  
                "lm_owner": "{{ owner_n_number }}",  
                "lm_sbu": "{{ business_unit_key }}",  
                "lm_troux_uid": "{{ troux_id }}"  
            }  
        ),  
        spec=k8s.V1PodSpec(  
            containers=[  
                k8s.V1Container(  
                    name="base-airflow-image",  
                    image="{{ image }}",  
                    image_pull_policy="IfNotPresent",  
                ),  
            ],  
        ),  
    ),  
}
```

What do you mean by rendering DAGs?

```
PODSPECS = {  
    "pod_override": k8s.V1Pod(  
        metadata=k8s.V1ObjectMeta(  
            labels={  
                "deployment_guid": "111-111",  
                "lm_app": "example-project",  
                "lm_app_env": "non-prod",  
                "lm_owner": "Bob",  
                "lm_sbu": "foreign-auto",  
                "lm_troux_uid": "private-id"  
            }  
        ),  
        spec=k8s.V1PodSpec(  
            containers=[  
                k8s.V1Container(  
                    name="base-airflow-image",  
                    image="matts/custom-image:latest",  
                    image_pull_policy="IfNotPresent",  
                ),  
            ],  
        ),  
    ),  
}
```

What do you mean by rendering DAGs?

```
PODSPECS = {  
  "pod_override": k8s.V1Pod(  
    metadata=k8s.V1ObjectMeta(  
      labels={  
        "deployment_guid": "{{ deployment_guid }}",  
        "lm_app": "{{ artifact_key }}",  
        "lm_app_env": "{{ environment }}",  
        "lm_owner": "{{ owner_n_number }}",  
        "lm_sbu": "{{ business_unit_key }}",  
        "lm_troux_uid": "{{ troux_id }}"  
      }  
    ),  
    spec=k8s.V1PodSpec(  
      containers=[  
        k8s.V1Container(  
          name="base-airflow-image",  
          image="{{ image }}",  
          image_pull_policy="IfNotPresent",  
        ),  
      ],  
    ),  
  ),  
}
```



```
PODSPECS = {  
  "pod_override": k8s.V1Pod(  
    metadata=k8s.V1ObjectMeta(  
      labels={  
        "deployment_guid": "111-111",  
        "lm_app": "example-project",  
        "lm_app_env": "non-prod",  
        "lm_owner": "Bob",  
        "lm_sbu": "foreign-auto",  
        "lm_troux_uid": "private-id"  
      }  
    ),  
    spec=k8s.V1PodSpec(  
      containers=[  
        k8s.V1Container(  
          name="base-airflow-image",  
          image="matts/custom-image:latest",  
          image_pull_policy="IfNotPresent",  
        ),  
      ],  
    ),  
  ),  
}
```

What do you mean by rendering DAGs?

```
PODSPECS = {
```

```
    "pod_override": k8s.V1Pod(  
        metadata=k8s.V1ObjectMeta(  
            labels={
```

```
                "deployment_guid": "{{ deployment_guid }}",
```

```
                "deployment_guid": "111-111",
```

```
                "app": "{{ artifact_key }}",
```

```
                "app_env": "{{ environment }}",
```

```
                "owner": "{{ owner_n_number }}",
```

```
                "business_unit": "{{ business_unit_key }}",
```

```
                "private_id": "{{ private_id }}"
```

```
            },
```

```
        ),
```

```
    spec=k8s.V1PodSpec(  
        containers=[
```

```
            k8s.V1Container(  
                name="base-airflow-image",
```

```
                image="{{ image }}",
```

```
                image_pull_policy="IfNotPresent",
```

```
            ),
```

```
        ],
```

```
    ),
```

```
},
```

my-dag-def.py.template



```
PODSPECS = {
```

```
    "pod_override": k8s.V1Pod(  
        metadata=k8s.V1ObjectMeta(  
            labels={
```

```
                "deployment_guid": "111-111",
```

```
                "deployment_guid": "111-111",
```

```
                "app": "example-project",
```

```
                "app_env": "dev-prod",
```

```
                "owner": "Bob",
```

```
                "business_unit": "foreign-auto",
```

```
                "private_id": "private-id"
```

```
            },
```

```
        ),
```

```
    spec=k8s.V1PodSpec(  
        containers=[
```

```
            k8s.V1Container(  
                name="base-airflow-image",
```

```
                image="matts/custom-image:latest",
```

```
                image_pull_policy="IfNotPresent",
```

```
            ),
```

```
        ],
```

```
    ),
```

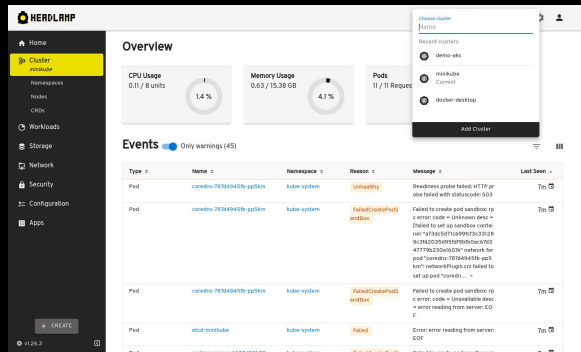
```
},
```

my-dag-def.py

What About Observability?



- DAG Render sidecar automatically detects changes in DAG code and writes out a .txt with the python errors detected in Airflow!
- DAG Render sidecar automatically converts .py.template files into their .py files using Jinja Templating!
- For k8s, we use **Headlamp**!
 - Users have scoped, read-only access to their namespace as well as the Managed Airflow namespace
 - Users can quickly check on logs of sidecars to debug problems



Benefits of Personal Airflow



Simulation of Data
Access Patterns

Same level of
access in
Personal Airflow
as in Managed
Airflow

Quicker development
turnaround

Iterate quickly on
a single DAG
without needing
to deploy to
validate/verify

Team Collaboration
Streamlined!

Improves
team-wide
development on
the same
repository

Clear insight into
"bugs" DAG
limitations prior to
deploying

Simulation of
Kubernetes
Environment with
complete observability

Integration with
Headlamp for
debugging

Integration with
Datadog for
performance
metrics



Questions?

Matt Davis
[linkedin.com/in/matt-davis17](https://www.linkedin.com/in/matt-davis17)
Matt.Davis17@gmail.com

Matt Koski
[linkedin.com/in/matthew-koski-30440331](https://www.linkedin.com/in/matthew-koski-30440331)
maccam912@gmail.com